

EFFICIENT IMPLEMENTATION OF A CLASS OF PRECONDITIONED CONJUGATE GRADIENT METHODS*

STANLEY C. EISENSTAT†

Abstract. The preconditioned conjugate gradient (PCG) method is an effective means for solving systems of linear equations where the coefficient matrix is symmetric and positive definite. The incomplete LDL' factorizations are a widely used class of preconditionings, including the SSOR, Dupont-Kendall-Rachford, generalized SSOR, ICCG(0), and MICCG(0) preconditionings. The efficient implementation of PCG with a preconditioning from this class is discussed.

Key words. incomplete factorization, preconditioned conjugate gradient methods, preconditioning

1. Introduction. Consider the system of N linear equations

$$(1) \quad Ax = b,$$

where the coefficient matrix A is symmetric and positive definite. When A is large and sparse, the preconditioned conjugate gradient (PCG) method is an effective means for solving (1) [2], [4], [5], [9], [13]. Given an initial guess x_0 , we generate a sequence $\{x_k\}$ of approximations to the solution x as follows:

$$(2a) \quad r_0 = b - Ax_0$$

$$(2b) \quad \text{Solve } Mr'_0 = r_0 \text{ and set } p_0 = r'_0$$

FOR $k = 0$ STEP 1 UNTIL Convergence DO

$$(2c) \quad a_k = (r_k, r'_k) / (p_k, Ap_k)$$

$$(2d) \quad x_{k+1} = x_k + a_k p_k$$

$$(2e) \quad r_{k+1} = r_k - a_k Ap_k$$

$$(2f) \quad \text{Solve } Mr'_{k+1} = r_{k+1}$$

$$(2g) \quad b_k = (r_{k+1}, r'_{k+1}) / (r_k, r'_k)$$

$$(2h) \quad p_{k+1} = r'_{k+1} + b_k p_k.$$

The effect of the preconditioning matrix M is to increase the rate of convergence of the basic conjugate gradient method of Hestenes and Stiefel [11]. The number of multiply-adds per iteration is just $5N$, plus the number required to form Ap_k , plus the number required to solve $Mr'_k = r_k$.

One widely used class of preconditionings are the incomplete LDL' factorizations

$$(3) \quad M = (\tilde{D} + L)\tilde{D}^{-1}(\tilde{D} + L)',$$

where $A \equiv L + D + L'$, L is strictly lower triangular and D and $\tilde{D}$ are positive diagonal. This class includes the SSOR [9], Dupont-Kendall-Rachford [7], generalized SSOR [1], ICCG(0) [13] and MICCG(0) [10] preconditionings. If we let $NZ(A)$ denote the number of nonzero entries in the matrix A , a straightforward implementation of

* Received by the editors October 30, 1980. This research was supported in part by the U.S. Office of Naval Research under grant N00014-76-C-0277.

† Research Center for Scientific Computation, Department of Computer Science, Yale University, New Haven, Connecticut 06520.

PCG with a preconditioning from this class¹ would require $6N + 2\text{NZ}(A)$ multiply-adds per iteration.²

In this brief note, we show how to reduce the work to $8N + \text{NZ}(A)$ multiply-adds, asymptotically half as many as the straightforward implementation.³ We give details in § 2 and consider some generalizations in § 3.

2. Implementation. The linear system (1) can be restated in the form

$$(4) \quad [(\tilde{D} + L)^{-1}A(\tilde{D} + L)^{-t}][(\tilde{D} + L)^t x] = [(\tilde{D} + L)^{-1}b]$$

or

$$(5) \quad \hat{A}\hat{x} = \hat{b}.$$

But applying PCG to (1) with $M = (\tilde{D} + L)\tilde{D}^{-1}(\tilde{D} + L)^t$ is equivalent to applying PCG to (5) with $\tilde{M} = \tilde{D}^{-1}$ and setting $x = (\tilde{D} + L)^{-t}\hat{x}$.⁴ If we update x instead of $\hat{x}$ at each iteration, algorithm (2) becomes:

$$(6a) \quad \hat{r}_0 = (\tilde{D} + L)^{-1}(\hat{b} - \hat{A}x_0)$$

$$(6b) \quad \hat{p}_0 = \hat{r}'_0 = \tilde{D}\hat{r}_0$$

FOR $k = 0$ STEP 1 UNTIL Convergence DO

$$(6c) \quad \hat{a}_k = (\hat{r}_k, \hat{r}'_k) / (\hat{p}_k, \hat{A}\hat{p}_k)$$

$$(6d) \quad x_{k+1} = x_k + \hat{a}_k(\tilde{D} + L)^{-t}\hat{p}_k$$

$$(6e) \quad \hat{r}_{k+1} = \hat{r}_k - \hat{a}_k\hat{A}\hat{p}_k$$

$$(6g) \quad \text{Compute } \hat{r}'_{k+1} = \tilde{D}\hat{r}_{k+1}$$

$$(6g) \quad \hat{b}_k = (\hat{r}_{k+1}, \hat{r}'_{k+1}) / (\hat{r}_k, \hat{r}'_k)$$

$$(6h) \quad \hat{p}_{k+1} = \hat{r}'_{k+1} + \hat{b}_k\hat{p}_k$$

$\hat{A}\hat{p}_k$ can be computed efficiently by taking advantage of the following identity:

$$(7) \quad \begin{aligned} \hat{A}\hat{p}_k &= (\tilde{D} + L)^{-1}[(\tilde{D} + L) + (\tilde{D} + L)^t - (2\tilde{D} - D)](\tilde{D} + L)^{-t}\hat{p}_k \\ &= (\tilde{D} + L)^{-t}\hat{p}_k + (\tilde{D} + L)^{-1}[\hat{p}_k - K(\tilde{D} + L)^{-t}\hat{p}_k], \end{aligned}$$

where $K \equiv 2\tilde{D} - D$. Thus

$$(8a) \quad \hat{r}_k = (\tilde{D} + L)^{-t}\hat{p}_k,$$

$$(8b) \quad \hat{A}\hat{p}_k = \hat{r}_k + (\tilde{D} + L)^{-1}(\hat{p}_k - K\hat{r}_k),$$

which requires $2N + \text{NZ}(A)$ multiply-adds. $\hat{r}_k$ can also be used to update x_k in (6d), so that the total cost for each PCG iteration is just $8N + \text{NZ}(A)$ multiply-adds⁵ (plus

¹ Writing M as $(\tilde{D} + L)(I + \tilde{D}^{-1}L^t)$, we solve $Mr'_k = r_k$ by solving the triangular systems

$$(\tilde{D} + L)t_k = r_k, \quad (I + \tilde{D}^{-1}L^t)r'_k = t_k.$$

² $2N$ (respectively, N) multiply-adds can be saved by symmetrically scaling the problem to make $\tilde{D} = I$ (respectively, $D = I$).

³ A similar speedup for pairs of linear iterative methods is given in [6].

⁴ Both are equivalent to applying the basic conjugate gradient method to the preconditioned system

$$\bar{A}\bar{x} \equiv [\tilde{D}^{1/2}(\tilde{D} + L)^{-1}A(\tilde{D} + L)^{-t}\tilde{D}^{1/2}][\tilde{D}^{-1/2}(\tilde{D} + L)^t x] = [\tilde{D}^{1/2}(\tilde{D} + L)^{-1}b] \equiv \bar{b}$$

(see [4, pp. 58–59]).

⁵ Again, $3N$ multiply-adds can be saved by symmetrically scaling the problem so that $\tilde{D} = I$.

an additional N words of storage), versus $6N + 2NZ(A)$ for the straightforward implementation.

3. Generalizations. The approach presented in § 2 extends immediately to preconditionings of the form

$$(9) \quad M = (\tilde{D} + L)\tilde{S}^{-1}(\tilde{D} + L)^t,$$

where $\tilde{S}$ is positive diagonal. Moreover, if we take $K \equiv \tilde{D} + \tilde{D}^t - D$ in (7) and (8), then $\tilde{D}$ need not be diagonal or even symmetric. In this case, $\tilde{D}$ would reflect changes to both the diagonal and offdiagonal entries of A in generating an incomplete factorization. If we assume that only the nonzero entries of A are changed, i.e., that $(K)_{ij}$ is nonzero only if $(A)_{ij}$ is nonzero, then the operation count is $7N + NZ(A) + NZ(K)$.

Another application is to preconditioning nonsymmetric systems. Let

$$(10) \quad M = (\tilde{D} + L)\tilde{S}^{-1}(\tilde{D} + U)$$

be an incomplete LDU factorization of a nonsymmetric matrix A , where $A \equiv L + D + U$, L (respectively, U) is strictly lower (respectively, upper) triangular and D and $\tilde{S}$ are diagonal. Then a number of authors have proposed solving the linear system $Ax = b$ by using the conjugate gradient method to solve the normal equations for one of the preconditioned systems

$$(11a) \quad \hat{A}_1 \hat{x} \equiv [\tilde{S}(\tilde{D} + L)^{-1}A(\tilde{D} + U)^{-1}][(\tilde{D} + U)x] = [\tilde{S}(\tilde{D} + L)^{-1}b] \equiv \hat{b}$$

(see [12]) and

$$(11b) \quad \hat{A}_2 x \equiv [(\tilde{D} + U)^{-1}\tilde{S}(\tilde{D} + L)^{-1}A]x = [(\tilde{D} + U)^{-1}\tilde{S}(\tilde{D} + L)^{-1}b] \equiv \hat{b}$$

(see [14], [3]). $\hat{A}_2 \hat{p}$ can be computed as

$$(12) \quad \hat{A}_2 \hat{p} = (\tilde{D} + U)^{-1}\tilde{S}[\hat{p} + (\tilde{D} + L)^{-1}(D + U - \tilde{D})\hat{p}]$$

in $4N + NZ(L) + 2NZ(U)$ multiply-adds, whereas $\hat{A}_1 \hat{p}$ can be computed as

$$(13a) \quad \hat{r} = (\tilde{D} + U)^{-1}\hat{p},$$

$$(13b) \quad \hat{A}_1 \hat{p} = \tilde{S}[\hat{r} + (\tilde{D} + L)^{-1}(\hat{p} - (2\tilde{D} - D)\hat{r})]$$

in $4N + NZ(L) + NZ(U)$ multiply-adds. Thus the preconditioning (11a) would be more efficient per iteration, although more iterations might be required to achieve comparable accuracy [14].⁶

REFERENCES

- [1] O. AXELSSON, *A generalized SSOR method*, BIT, 13 (1972), pp. 443–467.
- [2] ———, *On preconditioning and convergence acceleration in sparse matrix problems*, Technical Report 74-10, CERN, May, 1974.
- [3] ———, *Conjugate gradient type methods for unsymmetric and inconsistent systems of linear equations*, Linear Algebra Appl., 29 (1980), pp. 1–16.
- [4] RATI CHANDRA, *Conjugate Gradient Methods for Partial Differential Equations*, PhD thesis, Department of Computer Science, Yale University, New Haven, CT, 1978.
- [5] PAUL CONCUS, GENE H. GOLUB AND DIANNE P. O'LEARY, *A generalized conjugate gradient method for the numerical solution of elliptic partial differential equations*, in *Sparse Matrix Computations*, James R. Bunch and Donald J. Rose, eds., Academic Press, New York, 1976, pp. 309–332.

⁶ The same would be true if a generalized conjugate residual method such as Orthomin [15], [8] were used to solve (11a) or (11b).

- [6] V. CONRAD AND Y. WALLACH, *Alternating methods for sets of linear equations*, Numer. Math., 32 (1979), pp. 105–108.
- [7] TODD DUPONT, RICHARD P. KENDALL AND H. H. RACHFORD JR., *An approximate factorization procedure for solving self-adjoint elliptic difference equations*, SIAM J. Numer. Anal., 5 (1968), pp. 559–573.
- [8] S. C. EISENSTAT, H. ELMAN, M. H. SCHULTZ AND A. H. SHERMAN, *Solving approximations to the convection diffusion equation*, in Society of Petroleum Engineers of AIME, Proceedings of the Fifth Symposium on Reservoir Simulation, February, 1979, pp. 127–132.
- [9] D. J. EVANS, *The analysis and application of sparse matrix algorithms in the finite element method*, in The Mathematics of Finite Elements and Applications, J. R. Whiteman, ed., Academic Press, New York, 1973, pp. 427–447.
- [10] IVAR GUSTAFSSON, *A class of first order factorization methods*, BIT, 18 (1978), pp. 142–156.
- [11] MAGNUS R. HESTENES AND EDUARD STIEFEL, *Methods of conjugate gradients for solving linear systems*, J. Res. National Bureau of Standards, 49 (1952), pp. 409–436.
- [12] DAVID S. KERSHAW, *The incomplete Cholesky-conjugate gradient method for the solution of systems of linear equations*, J. Comput. Phys., 26 (1978), pp. 43–65.
- [13] J. A. MEIJERINK AND H. A. VAN DER VORST, *An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix*, Math. Comp., 31 (1977), pp. 148–162.
- [14] M. PETRAVIC AND G. KUO-PETRAVIC, *An ILUCG algorithm which minimizes in the Euclidean norm*, J. Comput. Phys., 32 (1979), pp. 263–269.
- [15] P. K. W. VINSOME, *Orthomin, an iterative method for solving sparse sets of simultaneous linear equations*, in Society of Petroleum Engineers of AIME, Proceedings of the Fourth Symposium on Reservoir Simulation, February, 1976, pp. 150–159.